REPORT

Forest Fire Detection & Spread Prediction System

Team Members:

Don Varkey Joshy
John K Simson
Avadhanam Hasini

Milestone – 2

Historical Data Processing & ML Model Development

Overview

Milestone 2 focuses on the development and training of a machine learning model capable of classifying fire risk levels based on environmental sensor and weather data. A synthetic dataset calibrated to real Uttarakhand wildfire conditions is generated to train a Gradient Boosting Classifier. The trained model and its label encoder are serialized to disk as .pkl files and subsequently used by the live prediction pipeline in Milestone 3. Additionally, the prediction script demonstrates how the trained model runs inference against live data fetched from ThingSpeak and OpenWeatherMap, producing structured risk assessments for each prediction cycle.

Objectives

- Generate a synthetic dataset calibrated to Uttarakhand wildfire environmental conditions.
- Engineer features by fusing IoT sensor readings with OpenWeatherMap API weather data.
- Train a Gradient Boosting Classifier to classify fire spread risk into four levels: LOW, MEDIUM, HIGH, and FIRE.
- Evaluate the trained model using accuracy, confusion matrix, and classification report metrics.
- Serialize the trained model and label encoder as .pkl files for use in live prediction.
- Run live inference using ThingSpeak sensor data combined with real-time weather API values.
- Export all prediction results to CSV and Excel files for downstream reporting and analysis.

Section 1: Synthetic Dataset Generation

Since real historical fire data for Uttarakhand at the required sensor-level granularity is unavailable publicly, a synthetic dataset is generated programmatically. The dataset reflects four fire-risk scenarios with class weights that mirror real-world frequency observations from the Uttarakhand 2024 fire season.

Scenario Distribution

Scenario	Weight	Temp Range	Humidity Range	Gas / Flame
Normal (LOW)	35%	14 – 26°C	55 – 80%	0 / 0
Medium Risk	30%	28 – 36°C	20 – 45%	Random / 0

High Risk	20%	36 – 46°C	8 – 20%	1 / 0
FIRE	15%	46 – 65°C	4 – 12%	1 / 1

For each scenario, sensor and API values are generated with realistic offsets to simulate the disagreement that occurs between a locally mounted DHT11 sensor and a regional weather station. Humidity values are clamped to the physical range of 4–100%.

Dataset Columns

Column	Description
Sensor_Temp	Raw temperature reading from the DHT11 sensor (°C)
Sensor_Hum	Raw humidity reading from the DHT11 sensor (%)
API_Temp	Temperature from OpenWeatherMap for the nearest station (°C)
API_Hum	Humidity from OpenWeatherMap for the nearest station (%)
Wind_Speed	Wind speed from the weather API (km/h)
Gas	MQ-2 binary output: 0 = no gas, 1 = gas/smoke detected
Flame	Flame sensor binary output: 0 = no flame, 1 = flame detected
Temp_Diff	Absolute difference between Sensor_Temp and API_Temp
Hum_Diff	Absolute difference between Sensor_Hum and API_Hum
Final_Temp	Fused temperature: average of Sensor_Temp and API_Temp
Final_Hum	Fused humidity: average of Sensor_Hum and API_Hum
Dryness_Index	Final_Temp minus Final_Hum — primary fire-risk indicator
Fire_Spread_Level	Target label: LOW, MEDIUM, HIGH, or FIRE

Label Assignment Rules

Labels are assigned deterministically based on sensor readings and derived features:

```
if flame == 1 and gas == 1:       label = 'FIRE'
elif dryness_index > 28
     and wind_speed > 28:          label = 'HIGH'
elif dryness_index > 12
     or  wind_speed > 15:          label = 'MEDIUM'
else:                              label = 'LOW'
```

Dataset Generation Code

```
def generate_dataset(num_rows=10000):
    random.seed(42)
    np.random.seed(42)
    rows = []
    for _ in range(num_rows):
        scenario = np.random.choice(
```

```
            ['normal', 'medium_risk', 'high_risk', 'fire'],
            p=[0.35, 0.30, 0.20, 0.15]
        )
        # ... generate sensor_temp, sensor_hum, api_temp,
        #     api_hum, wind_speed, gas, flame per scenario
        api_hum       = max(4, min(100, api_hum))
        final_temp    = round((sensor_temp + api_temp) / 2, 2)
        final_hum     = round((sensor_hum  + api_hum)  / 2, 2)
        dryness_index = round(final_temp - final_hum, 2)
        temp_diff     = round(abs(sensor_temp - api_temp), 2)
        hum_diff      = round(abs(sensor_hum  - api_hum), 2)
        # assign label using rules above
        rows.append({ ... })
    return pd.DataFrame(rows)
```

Section 2: Feature Engineering

Raw sensor and API values are combined into engineered features before being passed to the model. This fusion step reduces noise from individual sensor readings and produces a more reliable input representation.

Feature	Role in Model
Final_Temp	Fused temperature — reduces local sensor noise
Final_Hum	Fused humidity — reduces local sensor noise
Dryness_Index	Primary fire-risk signal; values above 28 indicate dangerous conditions
Wind_Speed	Accelerates fire spread; sourced directly from the weather API
Gas	Binary MQ-2 output; immediate chemical smoke/gas indicator
Flame	Binary flame sensor output; direct fire presence indicator
Temp_Diff	Sensor vs API disagreement; reflects micro-climate or sensor drift
Hum_Diff	Sensor vs API disagreement; flags unusual local humidity conditions

Section 3: Model Training

A Gradient Boosting Classifier is selected for this task due to its strong performance on tabular datasets, ability to handle mixed numerical and binary features, and built-in resistance to overfitting through iterative boosting with a learning rate.

Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size    = 0.2,
    random_state = 42,
    stratify     = y_encoded   # preserves class ratio
)
```

Model Configuration

```
model = GradientBoostingClassifier(
    n_estimators  = 150,   # number of boosting stages (trees)
    learning_rate = 0.1,   # shrinks each tree's contribution
    max_depth     = 4,     # max depth of each decision tree
    random_state  = 42
)
model.fit(X_train, y_train)
```

Label Encoding

The target column Fire_Spread_Level (strings: LOW, MEDIUM, HIGH, FIRE) is encoded to integers using scikit-learn's LabelEncoder before training. The encoder is fitted on the full label set and saved alongside the model so that predictions can be decoded back to their original string labels during inference.

```
le        = LabelEncoder()
y_encoded = le.fit_transform(df['Fire_Spread_Level'])
# Encoded classes example:
# FIRE=0, HIGH=1, LOW=2, MEDIUM=3  (alphabetical order)
```

Section 4: Model Evaluation

After training, the model is evaluated on the held-out test set (20% of the data). Three metrics are reported: overall accuracy, a per-class confusion matrix, and a full classification report containing precision, recall, and F1-score for each fire-risk class.

Evaluation Code

```
y_pred   = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f'Model Accuracy: {accuracy * 100:.2f}%')

# Confusion matrix
cm_df = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=cm_labels, columns=cm_labels
)

# Classification report
print(classification_report(
    y_test, y_pred,
    target_names=le.inverse_transform(sorted(set(y_test)))
))
```

Section 5: Saving Model Artifacts

After training and evaluation, the model and label encoder are serialized using Python's pickle module. These files are required by the live prediction script (predict_live.py) in Milestone 3.

```
MODEL_PATH   = 'forest_fire_model.pkl'
ENCODER_PATH = 'label_encoder.pkl'
```

```
with open(MODEL_PATH,   'wb') as f:
    pickle.dump(model, f)

with open(ENCODER_PATH, 'wb') as f:
    pickle.dump(le, f)
```

File	Contents
forest_fire_model.pkl	Trained GradientBoostingClassifier object
label_encoder.pkl	Fitted LabelEncoder — maps integers back to class names (FIRE, HIGH, LOW, MEDIUM)

Section 6: Live Prediction Pipeline

The predict_live.py script loads the trained model artifacts and runs a configurable number of prediction cycles. Each cycle fetches the latest sensor data from ThingSpeak, retrieves live weather from OpenWeatherMap, fuses the two sources, and passes the feature vector through the Gradient Boosting model.

Configuration Parameters

Parameter	Value / Description
THINGSPEAK_CHANNEL_ID	3333848 — matches the ESP32 Arduino sketch
THINGSPEAK_READ_KEY	Read API Key from ThingSpeak channel settings
WEATHER_API_KEY	OpenWeatherMap free-tier API key
WEATHER_CITY	Nainital (representative Kumaon monitoring station)
WEATHER_UNITS	metric — returns temperatures in Celsius
NUM_CYCLES	10 — number of prediction loops to run

Data Fusion Logic

Sensor readings from ThingSpeak and weather values from OpenWeatherMap are combined using simple averaging. The resulting fused values and derived metrics form the final feature vector fed to the model:

```
final_temp    = (sensor_temp + api_temp) / 2
final_hum     = (sensor_hum  + api_hum)  / 2
dryness_index = final_temp - final_hum
temp_diff     = abs(sensor_temp - api_temp)
hum_diff      = abs(sensor_hum  - api_hum)
```

Prediction & Alert Logic

After the model produces a class prediction, a hard safety override rule is applied: if both the gas sensor and flame sensor simultaneously read HIGH (binary 1), the system immediately raises a

FIRE DETECTED alert regardless of the model output. This ensures no active fire condition is missed due to edge cases in the training distribution.

```
pred_label = le.inverse_transform(model.predict(fv))[0]

# Hard safety override
if flame == 1 and gas == 1:
    fire_alert = 'FIRE DETECTED'
else:
    fire_alert = 'NORMAL'
```

Prediction Output

Each prediction cycle generates a structured output block in the console. The block is divided into four sections: data source metadata, raw sensor and weather values, comparison between sensor and API readings, fused values fed into the model, and the final decision output. The screenshot below shows a sample output from Prediction Cycle #10:

```
── PREDICTION CYCLE #10 ──

Sensor Data : ThingSpeak LIVE — channel 3333848
Weather     : OpenWeatherMap LIVE (Coord: 29.3919, 79.4542)
Timestamp   : 2026-04-09T17:51:23Z

── RAW VALUES ──
Sensor Temperature    : 16.0°C
Sensor Humidity       : 67.0%
API Temperature       : 4.1°C
API Humidity          : 58%
Wind Conditions       : 9.6 km/h from NE
Gas (MQ2)             : HIGH  [Binary: 1]
Flame Detected        : NO    [Binary: 0]

── COMPARISON  (Sensor vs API) ──
Temperature Diff      : 11.9°C
Humidity Diff         : 9.0%

── FUSED VALUES  (fed into ML model) ──
Final Temperature     : 10.05°C
Final Humidity        : 62.5%
Dryness Index         : -52.45

── DECISION OUTPUT ──
Fire Alert            : ⚠️GAS/SMOKE DETECTED
Fire Spread Level     : 🟢 LOW
Spread Path           : None (Low Risk)
```

Session Summary & Output Files

After all prediction cycles complete, results are aggregated into a session summary table and saved to two output formats:

- forest_fire_predictions.csv — comma-separated file containing all cycle results, suitable for pandas, Excel, and Power BI ingestion.
- forest_fire_predictions.xlsx — formatted Excel workbook containing Cycle, Timestamp, Sensor_Temp, Sensor_Hum, Gas, Flame, Fire_Alert, and Spread_Level columns.

Cycle	Timestamp	Sensor_Temp	Sensor_Hum	Gas	Flame	Fire_Alert	Spread_Level
1	2026-04-09T17:42:08Z	16.0	67.0	1	0	NORMAL	LOW
2	2026-04-09T17:42:08Z	16.0	67.0	1	0	NORMAL	LOW
...	...	...	...	...	...	...	...
10	2026-04-09T17:51:23Z	16.0	67.0	1	0	NORMAL	LOW

Outcome

At the end of Milestone 2, the following deliverables were achieved:

- A 10,000-row synthetic dataset was generated with Uttarakhand calibrated environmental conditions across four fire-risk scenarios.
- Feature engineering produced eight model input features by fusing sensor and API data and computing derived metrics.
- A Gradient Boosting Classifier was successfully trained with 150 estimators, achieving high classification accuracy on the test set.
- Model performance was validated using a confusion matrix and classification report.
- The trained model and label encoder were saved as forest_fire_model.pkl and label_encoder.pkl respectively.
- The live prediction pipeline was implemented and tested, successfully reading from ThingSpeak and OpenWeatherMap APIs and generating fire risk predictions for 10 consecutive cycles.
- All prediction results were exported to CSV and Excel files for use in Milestone 4 dashboard visualization.

This milestone delivers the core ML intelligence of the system. The trained model bridges raw IoT sensor data with actionable fire-risk predictions, establishing the foundation for real-time decision support in forest fire management.